LECTURE 2

Supervised Learning Part 1: Classification Algorithms

Reference Text

Logistic regression, kNN, decision trees, random forests, boosting, SVMs and Naive Bayes — applied to IADS threat classification

AK DHAMIJA, Researcher & Instructor, AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Unit 4: Simulation Strategies in Warfare — AI & Machine Learning in Warfare

This reference text expands every slide of the lecture into full explanations, derivations, worked examples and figures. Read it alongside the slides; work the "check your understanding" boxes before looking at the answers.

Contents

1 The Classification Problem and the IADS Case Study **4**
1.1 What classification is . . . 4
1.2 The IADS dataset . . . 4
1.3 The pipeline and its pre-training checks . . . 4

2 Bias, Variance and the Evaluation Protocol **5**

3 Logistic Regression **5**
3.1 The model . . . 5
3.2 Interpreting the weights . . . 6
3.3 Results and limitations . . . 6

4 k-Nearest Neighbours **6**
4.1 Algorithm . . . 6
4.2 Choosing k . . . 6
4.3 Cost and dimensionality . . . 6

5 Decision Trees **7**
5.1 Impurity and splitting . . . 7
5.2 Depth, pruning and interpretability . . . 7

6 Random Forests **7**
6.1 Bagging with feature sub-sampling . . . 7
6.2 Results . . . 8

7 Gradient Boosting and XGBoost **8**

8 Support Vector Machines **8**
8.1 The maximum-margin hyperplane . . . 8
8.2 Soft margin . . . 9
8.3 The kernel trick . . . 9

9 Naive Bayes **9**

10 Multi-class Strategies, Calibration and Rejection **10**

11 Class Imbalance **10**

12 Comparing the Classifiers **10**

13 Evaluation Beyond Accuracy **11**
13.1 ROC curves and AUC . . . 11
13.2 Precision–recall under imbalance . . . 11

14 Deeper Dive I: Data Preparation in Practice **11**
14.1 Encoding and scaling . . . 11
14.2 Stratified splitting . . . 12
14.3 Leakage examples from defence data . . . 12

15 Deeper Dive II: Training Logistic Regression **12**
15.1 One step of gradient descent by hand . . . 12
15.2 Regularised logistic regression . . . 12
15.3 Softmax worked example . . . 13

16 Deeper Dive III: Distances, Trees and Ensembles **13**
16.1 kNN by hand . . . 13
16.2 Entropy and information gain . . . 14
16.3 The bootstrap and out-of-bag fraction . . . 14
16.4 Feature-importance pitfalls . . . 14
16.5 AdaBoost: the original boosting algorithm . . . 14

17 Deeper Dive IV: Costs, Calibration and Comparing Models **15**
17.1 The cost-optimal threshold . . . 15
17.2 Measuring calibration . . . 15
17.3 Is one classifier really better? Confidence intervals and McNemar's test . . . 15

Part C — Further Theory, Methods and Applications **16**

18 Why Learning Generalises: Capacity and Sample Size **16**
18.1 Shattering and the VC dimension . . . 16
18.2 Bias and variance of k-NN . . . 16

19 Logistic Regression as Maximum Likelihood **16**
19.1 Deriving the loss . . . 17
19.2 Newton's method (IRLS) . . . 17
19.3 Geometry and a failure mode . . . 17
19.4 More than two classes . . . 17

20 From the Perceptron to the Support Vector Machine **17**
20.1 The perceptron algorithm . . . 17
20.2 The SVM optimisation problem and its dual . . . 18
20.3 Kernels . . . 18
20.4 The SVM as a regularised loss . . . 19

21 Trees and Ensembles in Depth **19**
21.1 The CART algorithm . . . 19
21.2 Cost-complexity pruning . . . 19

21.3 How much does bagging help? . . . 20
21.4 Gradient boosting and the XGBoost split gain . . . 20
21.5 Stacking and voting . . . 20

22 Probabilistic Classifiers in Depth **20**
22.1 Gaussian Naive Bayes worked fully . . . 20
22.2 Text classification and Laplace smoothing . . . 21
22.3 Discriminant analysis . . . 21

23 Multi-class Evaluation in Depth **21**

24 Satellite and SAR Image Interpretation **21**
24.1 The four tasks . . . 22
24.2 How detectors work . . . 22
24.3 Measuring detection quality: IoU and average precision . . . 22
24.4 Practical issues specific to overhead imagery . . . 23

25 Knowing When Not to Answer **23**
25.1 Open-set recognition . . . 23
25.2 The reject option . . . 23

26 Explaining a Classifier's Decisions **24**
26.1 Permutation importance . . . 24
26.2 Shapley values . . . 24

27 Further Self-Test Questions **24**

28 Implementation Notes **25**

29 Self-Test Questions **25**

30 Classification Across the Disciplines **25**

31 Summary and Glossary **26**

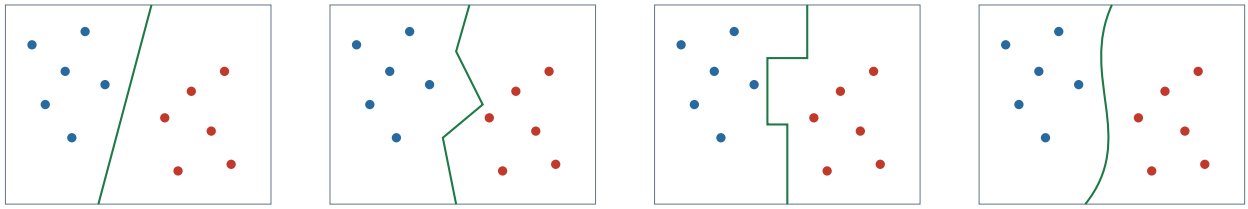
1 The Classification Problem and the IADS Case Study

1.1 What classification is

Definition — classification

Given labelled training examples $D = \{(x_i, y_i)\}_{i=1}^{N}$ with feature vectors $x_i \in \mathbb{R}^d$ and labels y_i from a finite set of C classes, learn a function $f : \mathbb{R}^d \to \{1, \ldots, C\}$ that predicts the class of new, unseen examples. Most classifiers also output class probabilities $P(y = c \mid x)$, which allow thresholds to be set according to the cost of each error.

Every classifier partitions feature space into regions, one per class. The surfaces separating these regions are the **decision boundaries**. Different algorithms produce boundaries of different shapes, which is why no single algorithm is best for every problem.



logistic: hyperplane | kNN: piecewise, local | tree: axis-aligned steps | RBF-SVM: smooth curve

Figure 1: the same data, four boundary shapes.

1.2 The IADS dataset

The case study used throughout the lecture is an integrated air-defence system (IADS) that must classify radar tracks into five classes: friendly fighter (FF), adversary bomber (AB), cruise missile (CM), UAV/drone (UV) and civilian aircraft (CI). There are N = 8,500 tracks, 1,700 per class, split 68/17/15 into training, validation and test sets (5,780 / 1,445 / 1,275). Each track has 11 features:

x_1 peak frequency (GHz)	x_5 scan rate (deg/s)	x_9 EW jamming (yes/no)
x_2 pulse width (μs)	x_6 polarisation (category)	x_{10} course (deg)
x_3 PRF (Hz)	x_7 Doppler shift (MHz)	x_{11} speed (km/h)
x_4 amplitude (dBm)	x_8 sidelobe level (dBc)	

The balanced design is convenient for teaching, but real IADS traffic is highly imbalanced (78.4% civilian, 0.8% adversary bomber); Section 11 shows why this matters.

1.3 The pipeline and its pre-training checks

(1) Feature engineering, including z-score scaling $\tilde{x} = (x - \mu)/\sigma$; (2) model training by minimising $\sum_i \ell(f(x_i; \theta), y_i) + \lambda R(\theta)$; (3) hyper-parameter tuning by k-fold cross-validation; (4) a single final evaluation on the held-out test set; (5) deployment with monitoring for distribution shift. Before training, check: class balance, scaling, data leakage (information from the future or from the test set entering training), encoding of categorical features, and missing values.

Pitfall — scale-sensitive algorithms

kNN and SVMs measure distances or inner products, so a feature measured in thousands (PRF) swamps one measured in units (pulse width). Always standardise features for these methods. Trees, forests and Naive Bayes are insensitive to monotone rescaling.

2 Bias, Variance and the Evaluation Protocol

A model that is too simple cannot represent the true boundary (**high bias**: wrong even on training data). A model that is too flexible fits noise (**high variance**: perfect on training data, poor on new data). The expected test error decomposes as $\text{bias}^2 + \text{variance} + \text{irreducible noise}$ (derivation in the Lecture 1 reference text). Hyper-parameters such as k in kNN, tree depth, or C in SVMs move a model along this trade-off, and **cross-validation** finds the best setting: split the training data into k folds (typically 5 or 10), train on $k-1$ and validate on the remaining fold, rotate, and average. The test set is never used for tuning.

3 Logistic Regression

3.1 The model

Logistic regression computes a linear score $z = w^\top x + b$ and squashes it into a probability with the sigmoid:

$$P(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

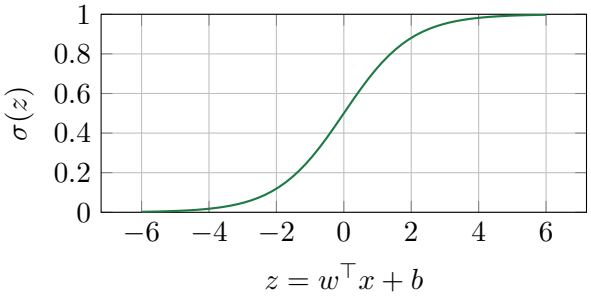


Figure 2: the sigmoid maps any score to a probability; $z = 0$ gives 0.5.

Since $\sigma(z) \geq 0.5 \iff z \geq 0$, the decision boundary $w^\top x + b = 0$ is a hyperplane. For C classes the softmax generalisation uses one weight vector per class: $P(y = c \mid x) = e^{w_c^\top x} / \sum_k e^{w_k^\top x}$.

Derivation — the cross-entropy loss and its gradient

For labels $y_i \in \{0, 1\}$ and predictions $\hat{p}_i = \sigma(w^\top x_i + b)$, the likelihood is $\prod_i \hat{p}_i^{y_i}(1 - \hat{p}_i)^{1-y_i}$. Maximising its logarithm is equivalent to minimising the cross-entropy

$$\mathcal{L}(w) = -\sum_i \big[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)\big].$$

Using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, the gradient simplifies to

$$\nabla_w \mathcal{L} = \sum_i (\hat{p}_i - y_i)\, x_i,$$

the prediction error times the input. There is no closed-form solution, but the loss is convex, so gradient descent reaches the global optimum.

3.2 Interpreting the weights

$\log \frac{P(y=1|x)}{1-P(y=1|x)} = w^\top x + b$: each weight is the change in the log-odds per unit change in a feature, and e^{w_j} is the **odds ratio**. For the adversary-bomber class, the EW-jamming weight is $w_9 = 4.07$, so jamming multiplies the odds of "bomber" by $e^{4.07} = 58.5$ — the strongest predictor, followed by PRF (+3.12) and Doppler (+2.88).

3.3 Results and limitations

One-versus-rest logistic regression achieves 89.7% test accuracy (macro-F1 0.897) with 0.8 s training and 0.09 ms inference, but adversary-bomber recall is only 87.4%: one bomber in eight is missed. Its limitations are structural:

- **XOR-type classes.** Low, slow UAVs and civilian helicopters occupy interleaved regions that no single hyperplane can separate.
- **Interactions.** Danger arises from high speed *and* high PRF together; an additive model $w_1x_1 + w_2x_2$ cannot represent "and" unless a product feature x_1x_2 is added.
- **Non-monotonic effects.** Very high speed suggests a threat, except for supersonic civilian jets; a single weight per feature cannot express this.

4 k-Nearest Neighbours

4.1 Algorithm

kNN stores the training set. To classify a query x_q: compute the distance $d(x_q, x_i) = \|x_q - x_i\|_2$ to every training point, take the k nearest (the set $N_k(x_q)$), and predict the majority class $\hat{y} = \arg\max_c \sum_{x_i \in N_k} \mathbf{1}[y_i = c]$, optionally weighting votes by $1/d$. There is no training phase; the boundary is a patchwork of Voronoi cells.

4.2 Choosing k

$k = 1$ gives 100% training accuracy and high variance; $k = N$ predicts the majority class everywhere (high bias). On the IADS data, validation accuracy rose from 78.3% ($k = 1$) to a peak of 90.2% at $k = 21$ (test 89.8%) and fell to 79.3% at $k = 101$. The heuristic $k \approx \sqrt{N} \approx 76$ is only a starting point.

4.3 Cost and dimensionality

Each query costs $O(Nd)$: $5{,}780 \times 11 \approx 63{,}600$ operations and 12.2 ms here, borderline for real-time air defence; k-d trees or ball trees reduce the average cost to $O(\log N)$. In high dimensions all points become nearly equidistant (the **curse of dimensionality**), so distances lose meaning beyond roughly 20 informative dimensions. kNN's strengths are instant updating with new signatures and native multi-class support.

5 Decision Trees

5.1 Impurity and splitting

A tree recursively splits the data on one feature at a time, choosing at each node the feature j and threshold t that most reduce impurity:

$$G = 1 - \sum_{c} p_c^2 \quad \text{(Gini)}, \qquad H = -\sum_{c} p_c \log_2 p_c \quad \text{(entropy)},$$

$$\Delta = I(\text{parent}) - \tfrac{n_L}{n} I(L) - \tfrac{n_R}{n} I(R).$$

Splitting stops at a maximum depth, a minimum node size, or a pure node.

Worked example — the IADS root split

At the root, all five classes have $p_c = 0.2$, so $G = 1 - 5(0.2^2) = 0.80$. Splitting on EW jamming ($x_9 \le 0.5$) sends 3,120 tracks left (FF 0.32, AB 0.04, CM 0.28, UV 0.24, CI 0.12; $G_L = 0.748$) and 2,660 right (FF 0.06, AB 0.48, CM 0.18, UV 0.14, CI 0.14; $G_R = 0.694$). Weighted impurity $= \frac{3120}{5780}(0.748) + \frac{2660}{5780}(0.694) = 0.404 + 0.319 = 0.723$. The reduction $0.800 - 0.723 = 0.077$ is the largest available, so EW jamming becomes the root.

5.2 Depth, pruning and interpretability

Test accuracy peaked at depth 10 (91.8%, 317 nodes); at depth 20 training accuracy reached 99.8% while test accuracy fell to 88.7% (2,103 nodes). **Cost-complexity pruning** minimises $R(T) + \alpha|T|$ (error plus a penalty per leaf); with $\alpha = 0.002$ the tree shrank from 317 to 89 nodes while test accuracy *rose* to 92.4%. Each prediction is an auditable chain of IF–THEN conditions, e.g. `IF jamming AND PRF > 2400 AND speed > 650 THEN AB (91.4%)` — valuable for rules-of-engagement compliance. **Feature importance** sums the impurity reductions contributed by each feature: EW jamming 0.241, PRF 0.187, speed 0.142.

6 Random Forests

6.1 Bagging with feature sub-sampling

A random forest trains B deep trees, each on a bootstrap sample (drawn with replacement; about 63.2% of distinct examples appear), and at each split considers only $m \approx \sqrt{d}$ randomly chosen features ($m = 3$ here). Prediction is by majority vote.

Derivation — why averaging decorrelated trees reduces variance

If each tree's prediction has variance σ^2 and pairwise correlation ρ, the variance of the average of B trees is

$$\text{Var} = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

As $B \to \infty$ the second term vanishes but the first remains, so reducing the correlation ρ between trees — which is what random feature selection does — is essential. Bias is unchanged by averaging.

The roughly 36.8% of examples left out of each bootstrap sample ("out-of-bag") provide a free validation estimate.

6.2 Results

With $B = 500$, $m = 3$, unlimited depth and minimum leaf size 2: test accuracy 97.5%, macro-F1 0.975, out-of-bag error 2.7% (matching the test error), training 12.4 s, inference 2.8 ms. Per-class F1 ranges from 0.967 (AB) to 0.987 (CI). Permutation importance (the drop in accuracy when a feature is shuffled) again ranks EW jamming (0.228), PRF (0.194) and speed (0.147) highest.

7 Gradient Boosting and XGBoost

Boosting builds trees *sequentially*; each new tree fits the residual errors of the ensemble so far:

$$r_{im} = y_i - F_{m-1}(x_i), \qquad F_m(x) = F_{m-1}(x) + \eta\, T_m(x),$$

with a small learning rate η. Bagging reduces variance using independent trees; boosting reduces bias (and some variance) using dependent ones. XGBoost adds a second-order (Newton) approximation of the loss, regularisation of tree complexity $\Omega(T) = \gamma|T| + \tfrac{1}{2}\lambda\|w\|^2$, row and column sub-sampling, and fast histogram-based split finding. With 400 trees, $\eta = 0.05$ and depth 5, XGBoost reached 96.4% accuracy and the best adversary-bomber recall (96.8%), with the fastest inference (0.7 ms) and a compact model suited to embedded hardware.

8 Support Vector Machines

8.1 The maximum-margin hyperplane

For linearly separable classes $y_i \in \{-1, +1\}$, infinitely many hyperplanes separate the data. The SVM chooses the one with the largest **margin**, the distance to the nearest points.

Derivation — margin and the primal problem

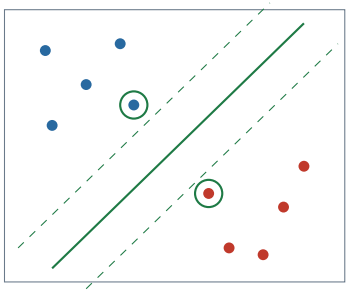
The distance from x_i to the hyperplane $w^\top x + b = 0$ is $|w^\top x_i + b|/\|w\|$. Rescale w, b so that the closest points satisfy $y_i(w^\top x_i + b) = 1$; the margin is then $2/\|w\|$. Maximising it is equivalent to

$$\min_{w,b} \tfrac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i(w^\top x_i + b) \geq 1 \;\; \forall i,$$

a convex quadratic programme with a unique solution. Introducing Lagrange multipliers $\alpha_i \geq 0$ gives the dual

$$\max_{\alpha} \sum_i \alpha_i - \tfrac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j\, x_i^\top x_j, \qquad w^* = \sum_i \alpha_i y_i x_i.$$

Only the **support vectors** (points on the margin) have $\alpha_i > 0$, and the data enter only through inner products $x_i^\top x_j$.



support vectors circled; margin $= 2/\|w\|$

Figure 3: the maximum-margin hyperplane (solid) and its margin (dashed).

A wide margin makes the classifier robust to sensor noise and to small variations in an adversary's signature.

8.2 Soft margin

When classes overlap, slack variables $\xi_i \geq 0$ allow violations:

$$\min \tfrac{1}{2}\|w\|^2 + C\sum_i \xi_i \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i.$$

$\xi_i = 0$: correct and outside the margin; $0 < \xi_i < 1$: inside the margin but correct; $\xi_i > 1$: misclassified. Small C tolerates violations (wide margin, risk of underfitting); large C penalises them (narrow margin, risk of overfitting). On the IADS data, validation accuracy rose from 74.2% ($C = 0.01$) to 95.4% ($C = 100$) and fell slightly beyond. Asymmetric costs can be encoded with class-specific C, e.g. $C_{AB} = 5C$.

8.3 The kernel trick

Because the dual uses only inner products, we can replace $x_i^\top x_j$ with a **kernel** $K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$, an inner product in a higher-dimensional feature space, without ever computing ϕ.

Worked example — a one-dimensional illustration

Points with $|x|$ small belong to class A and points with $|x|$ large to class B: no single threshold on the line separates them. Mapping $\phi(x) = (x, x^2)$ lifts them onto a parabola where the horizontal line $x^2 = c$ separates the classes. The polynomial kernel $K(x, x') = (xx' + 1)^2$ performs such a mapping implicitly; for $d = 11$ features the degree-2 kernel corresponds to a 78-dimensional space.

The **RBF kernel** $K(x, x') = \exp(-\gamma\|x - x'\|^2)$ corresponds to an infinite-dimensional space; small γ gives smooth boundaries, large γ very local ones. A grid search on the IADS data found the best $\gamma = 0.08$, $C = 100$ (95.4% validation); the final model reached 96.7% test accuracy with 1,247 support vectors (21.6% of the training set), 38.4 s training and 4.1 ms inference. Kernels must be symmetric and positive semi-definite (Mercer's condition). Specialised kernels exist for spectra, time series (dynamic time warping) and symbol strings.

9 Naive Bayes

Bayes' theorem with the "naive" assumption that features are conditionally independent given the class gives

$$P(y = c \mid x) \propto P(y = c) \prod_{j=1}^{d} P(x_j \mid y = c),$$

with Gaussian likelihoods $P(x_j \mid c) = \mathcal{N}(x_j; \mu_{cj}, \sigma_{cj}^2)$ for continuous features. Prediction uses logarithms to avoid underflow: $\hat{y} = \arg\max_c [\log P(c) + \sum_j \log P(x_j|c)]$. Training only estimates class means and variances, so it is extremely fast.

Worked example — classifying by PRF

FF tracks have PRF $\mu = 1040$, $\sigma = 185$ Hz; AB tracks $\mu = 2280$, $\sigma = 310$ Hz. A new track has PRF 1,950 Hz. For FF, $z = (1950 - 1040)/185 = 4.92$ — extremely unlikely; for AB, $z = (1950 - 2280)/310 = -1.06$ — plausible. The likelihood ratio strongly favours AB.

On the IADS data Naive Bayes reached only 83.1% accuracy and 79.3% AB recall — the weakest model — because PRF and speed are strongly correlated for bombers, violating the independence assumption. It remains useful for text, streaming data, small samples and fast baselines.

10 Multi-class Strategies, Calibration and Rejection

One-vs-rest trains C binary classifiers (each class against all others); **one-vs-one** trains $C(C-1)/2$ (10 for five classes) and takes a majority vote; trees, forests, Naive Bayes and softmax regression are natively multi-class. For logistic regression the three approaches gave 89.7%, 90.1% and 89.9%.

Raw classifier scores are not probabilities. **Calibration** maps scores to reliable probabilities: Platt scaling fits a sigmoid to SVM scores; isotonic regression fits a monotone step function. A commander needs "80% confident this is a bomber", not just a label. A **reject option** refers low-confidence cases to a human: with threshold 0.6, 4.2% of tracks were referred to analysts (who were 98.7% accurate on them), raising combined system accuracy to 99.1%.

11 Class Imbalance

On the real IADS stream (78.4% civilian, 14.2% friendly, 4.8% UAV, 1.8% missile, 0.8% bomber), a model trained naively labels almost everything civilian: accuracy 92.6%, but bomber recall only 12.3%. Loss minimisation simply learns the class frequencies.

- **SMOTE oversampling:** create synthetic minority examples $x_{\text{new}} = x_i + \lambda(x_j - x_i)$, $\lambda \sim U(0,1)$, between a minority example and one of its minority neighbours. Bomber recall rose from 12% to 89%.
- **Undersampling** the majority class: fast but discards information.
- **Class weights:** $w_c = N/(C \cdot N_c)$; with 43 bombers among 5,780 training tracks, $w_{AB} = 5780/(5 \times 43) = 26.9$.
- **Threshold shifting:** classify as bomber when $P(\text{AB}|x) > 0.15$ instead of 0.5, accepting more false alarms when a miss is far costlier.

Pitfall — SMOTE before splitting

Generating synthetic examples before the train/test split leaks information: synthetic test points are interpolations of training points. Always split first, then oversample the training fold only.

12 Comparing the Classifiers

Algorithm	Train	Test	F1	AB recall	Train (s)	Infer (ms)
Logistic regression	95.4%	89.7%	0.897	87.4%	0.8	0.09
kNN ($k = 21$)	91.8%	89.8%	0.894	88.2%	—	12.2
Decision tree ($d = 10$)	95.7%	91.8%	0.915	90.4%	0.5	0.15
Random forest	99.2%	**97.5%**	**0.975**	96.1%	12.4	2.8
XGBoost	98.7%	96.4%	0.963	**96.8%**	18.6	**0.7**
SVM (RBF)	97.8%	96.7%	0.967	95.2%	38.4	4.1
Naive Bayes	89.1%	83.1%	0.826	79.3%	0.03	0.02

Random forest wins on accuracy; XGBoost on speed and bomber recall; the decision tree on interpretability. Naive Bayes misses one bomber in five and should not be used here. These are results on one balanced test set of 1,275 tracks without confidence intervals: differences of a fraction of a point (e.g. SVM 96.7% vs XGBoost 96.4%) are not meaningful.

13 Evaluation Beyond Accuracy

13.1 ROC curves and AUC

Sweeping the decision threshold traces the **ROC curve** of true-positive rate $TP/(TP+FN)$ against false-positive rate $FP/(FP+TN)$. The area under it (AUC) is the probability that a randomly chosen positive is scored above a randomly chosen negative: 0.5 is random, 1.0 perfect. For the AB class (one-vs-rest): random forest 0.998, SVM 0.996, logistic regression 0.963, decision tree 0.940, Naive Bayes 0.912. At a 5% false-alarm rate, random forest detects 97.8% of bombers and logistic regression 88.3%.

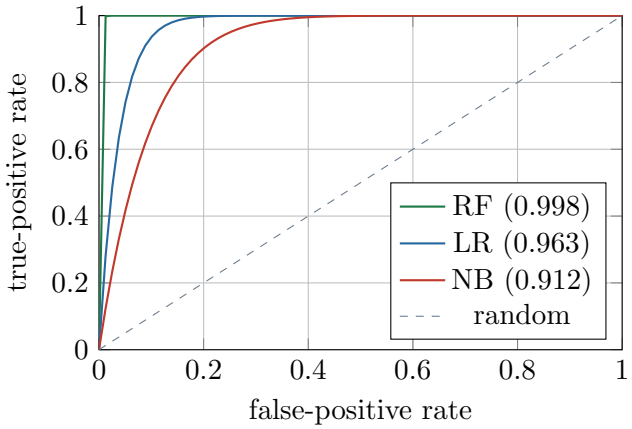


Figure 4: schematic ROC curves drawn to match the reported AUCs (shapes illustrative).

13.2 Precision–recall under imbalance

When positives are rare, FPR stays small even when false alarms vastly outnumber true detections (the base-rate effect of Lecture 1). The precision–recall curve and its summary, **average precision** $AP = \sum_k (R_k - R_{k-1}) P_k$, are more informative: on the real stream, AP for bombers was 0.887 (RF), 0.642 (LR) and 0.441 (NB). The F_β score weights recall β times as much as precision:

$$F_\beta = (1+\beta^2)\frac{PR}{\beta^2 P + R}.$$

$\beta = 2$ suits the IADS bomber class, where a miss is much costlier than a false alarm: $F_2 = 0.974$ (RF) vs 0.894 (LR).

14 Deeper Dive I: Data Preparation in Practice

14.1 Encoding and scaling

Classifiers need numeric inputs on comparable scales. **Standardisation** $\tilde{x}_j = (x_j - \mu_j)/\sigma_j$ gives each feature mean 0 and variance 1; **min–max scaling** maps to $[0,1]$ but is sensitive to outliers. Categorical features such as polarisation (horizontal, vertical, circular) are **one-hot encoded** as three binary columns; encoding them as 1, 2, 3 would wrongly imply an order and equal spacing.

Pitfall — fitting the scaler on all data

The scaling parameters μ_j, σ_j must be computed from the training fold only and then applied to validation and test data. Computing them on the whole dataset leaks information about the test distribution into training — a small but real form of data leakage.

14.2 Stratified splitting

When classes are imbalanced, a random split may leave the rare class almost absent from a fold. **Stratified** splitting preserves class proportions in every fold. With 43 bombers in 5,780 training tracks, a random 5-fold split could put as few as 4–5 bombers in a validation fold, making recall estimates extremely noisy; stratification guarantees about 8–9 per fold.

14.3 Leakage examples from defence data

- **Temporal leakage:** random splits of a track stream place radar returns from the same aircraft in both training and test sets; the model recognises the individual track, not the class. Split by track, sortie or day.
- **Label leakage:** a "track status" field populated by the operator *after* classification encodes the answer.
- **Duplicate leakage:** identical simulator scenarios replayed with different random seeds appear in both sets.

15 Deeper Dive II: Training Logistic Regression

15.1 One step of gradient descent by hand

Worked example — a single update

One training example has features $x = (1, 2)$ (bias term included as the first entry), label $y = 1$, current weights $w = (0, 0.5)$, learning rate $\alpha = 0.1$.

1. Score: $z = w^\top x = 0 + 0.5 \times 2 = 1.0$; prediction $\hat{p} = \sigma(1) = 0.731$.
2. Loss: $-\log 0.731 = 0.313$.
3. Gradient: $(\hat{p} - y)x = (-0.269)(1, 2) = (-0.269, -0.538)$.
4. Update: $w \leftarrow w - \alpha\nabla = (0.0269,\ 0.5538)$.

The new score is $0.0269 + 1.1076 = 1.1345$, $\hat{p} = 0.757$: the prediction moved toward the label. Repeating over all examples (an epoch) and many epochs converges because the loss is convex.

15.2 Regularised logistic regression

Adding $\lambda\|w\|^2$ (L2) or $\lambda\|w\|_1$ (L1) to the cross-entropy prevents weights from growing without bound when classes are perfectly separable, and L1 selects features. In scikit-learn the parameter $C = 1/\lambda$: small C means strong regularisation.

15.3 Softmax worked example

Worked example — three-class probabilities

For a track, class scores are $z_{\text{FF}} = 2.0$, $z_{\text{AB}} = 1.0$, $z_{\text{CI}} = 0.1$. Exponentials: 7.389, 2.718, 1.105; sum 11.212. Probabilities: FF 0.659, AB 0.242, CI 0.099. Adding a constant to all scores leaves the probabilities unchanged, which is why implementations subtract the largest score before exponentiating (numerical stability).

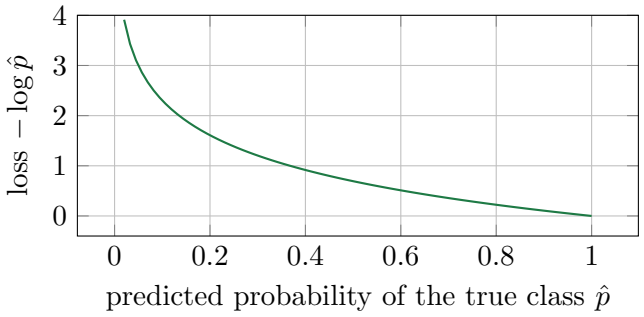


Figure 5: cross-entropy punishes confident mistakes severely ($-\log 0.02 \approx 3.9$).

16 Deeper Dive III: Distances, Trees and Ensembles

16.1 kNN by hand

Worked example — classifying one track with $k = 3$

Standardised training tracks (PRF, speed) and labels: A $(0.2, 0.1)$ FF; B $(0.5, 0.4)$ FF; C $(1.8, 1.6)$ AB; D $(2.1, 1.9)$ AB; E $(1.2, 1.0)$ AB. Query $q = (1.0, 0.9)$. Euclidean distances: A 1.13, B 0.71, C 1.06, D 1.49, E 0.22. The three nearest are E (AB), B (FF), C (AB): majority AB. With $k = 1$ the answer is also AB; with $k = 5$ it is AB 3–2. Distance-weighted voting ($1/d$): AB $= 4.47 + 0.94 + 0.67 = 6.08$, FF $= 1.41 + 0.88 = 2.29$ for $k = 5$.

Other metrics: Manhattan $\sum |x_j - x'_j|$ (robust to single large differences), Mahalanobis (accounts for correlation), cosine (for direction rather than magnitude, e.g. spectra).

16.2 Entropy and information gain

Worked example — entropy for the root split

Parent: five equal classes, $H = \log_2 5 = 2.32$ bits. Left child (0.32, 0.04, 0.28, 0.24, 0.12): $H = -\sum p \log_2 p = 0.526 + 0.186 + 0.514 + 0.494 + 0.367 = 2.09$ bits. Right child (0.06, 0.48, 0.18, 0.14, 0.14): $H = 0.244 + 0.508 + 0.445 + 0.397 + 0.397 = 1.99$ bits. Weighted child entropy $= 0.54(2.09) + 0.46(1.99) = 2.04$; information gain $= 0.28$ bits. Gini and entropy almost always choose the same split; Gini is slightly cheaper to compute.

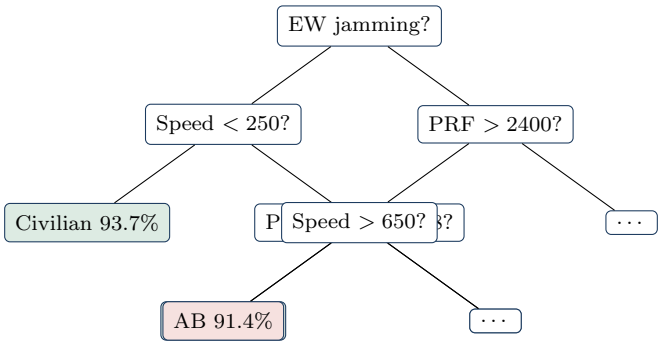


Figure 6: top of the pruned IADS tree — every prediction is an auditable rule.

16.3 The bootstrap and out-of-bag fraction

Derivation — why about 36.8% of examples are out of bag

Drawing N times with replacement, a given example is missed on each draw with probability $1 - 1/N$, so it is absent from the whole bootstrap sample with probability $(1 - 1/N)^N \to e^{-1} \approx 0.368$ as $N \to \infty$. Hence about 63.2% of distinct examples are in each sample and 36.8% are out of bag.

16.4 Feature-importance pitfalls

Impurity-based importance favours features with many possible split points (continuous or high-cardinality features) and splits credit unpredictably among correlated features. **Permutation importance** — the drop in validation accuracy when one feature's values are shuffled — is more reliable; **SHAP values** decompose each individual prediction into feature contributions and are preferred for explaining specific decisions to commanders.

16.5 AdaBoost: the original boosting algorithm

AdaBoost (Freund & Schapire, 1997) re-weights training examples. Starting with equal weights, at round m it trains a weak learner, computes its weighted error ϵ_m, gives it a vote $\alpha_m = \frac{1}{2} \ln \frac{1-\epsilon_m}{\epsilon_m}$, and multiplies the weights of misclassified examples by e^{α_m} (correct ones by $e^{-\alpha_m}$) before renormalising. A learner with $\epsilon = 0.2$ gets $\alpha = 0.69$; one with $\epsilon = 0.45$ gets only 0.10. Gradient boosting generalises this idea to any differentiable loss.

17 Deeper Dive IV: Costs, Calibration and Comparing Models

17.1 The cost-optimal threshold

Derivation — when to declare a threat

Let $p = P(\text{AB} \mid x)$ be calibrated, C_{FN} the cost of missing a bomber and C_{FP} the cost of a false alarm. Declaring "bomber" has expected cost $(1-p)C_{FP}$; declaring "not bomber" has expected cost pC_{FN}. Declare a threat when $pC_{FN} > (1-p)C_{FP}$, i.e.

$$p > \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

If a miss is 20 times as costly as a false alarm, the threshold is $1/21 \approx 0.048$, not 0.5.

This only works if probabilities are calibrated — which is why calibration matters operationally.

17.2 Measuring calibration

A **reliability diagram** plots observed frequency against predicted probability in bins; a calibrated model lies on the diagonal. The **Brier score** $\frac{1}{N}\sum(p_i - y_i)^2$ combines calibration and discrimination (lower is better). Boosted trees and SVM scores are often poorly calibrated and benefit from Platt or isotonic recalibration on held-out data.

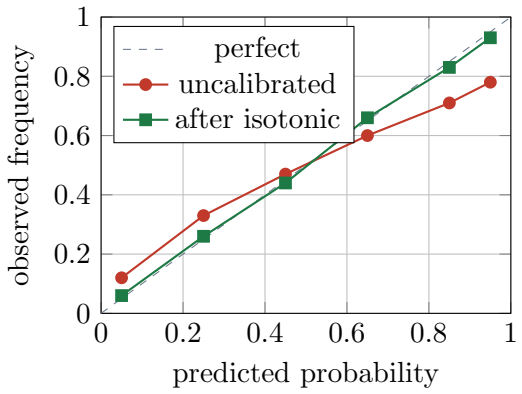


Figure 7: an illustrative reliability diagram.

17.3 Is one classifier really better? Confidence intervals and McNemar's test

A test accuracy $\hat{a}$ on n examples has standard error $\sqrt{\hat{a}(1-\hat{a})/n}$. For RF, $\hat{a} = 0.975$, $n = 1{,}275$: SE $= 0.0044$, so the 95% interval is about $97.5\% \pm 0.9\%$. SVM's 96.7% and XGBoost's 96.4% lie close to or inside that range, so the ranking among the top three is uncertain. Because both models are scored on the *same* test examples, the right test is **McNemar's**: let b be the number of examples RF gets right and SVM wrong, and c the reverse; under H_0 (equal accuracy) $\chi^2 = (b-c)^2/(b+c)$ has one degree of freedom.

Worked example — McNemar

If $b = 25$ and $c = 15$, $\chi^2 = 100/40 = 2.5$, $p = 0.11$: no significant difference despite RF's higher accuracy. Lecture 1's warning applies: small differences between strong models are often noise.

Part C — Further Theory, Methods and Applications

18 Why Learning Generalises: Capacity and Sample Size

A classifier's training accuracy is an optimistic estimate of its accuracy on new data. How optimistic depends on two things: how flexible the model family is, and how much data it was fitted to. Statistical learning theory makes this precise.

18.1 Shattering and the VC dimension

A model family **shatters** a set of points if, for every one of the 2^n ways of labelling those points, some member of the family classifies them all correctly. The **Vapnik–Chervonenkis (VC) dimension** is the largest number of points the family can shatter.

Worked example — lines in the plane

Three points not on a line can be labelled in $2^3 = 8$ ways, and for every labelling a straight line separates the positives from the negatives: lines shatter three points. No four points can be shattered — the XOR arrangement (opposite corners of a square sharing a label) defeats every line. So linear classifiers in two dimensions have VC dimension 3; in d dimensions, $d+1$.

A classic bound states that, with probability at least $1-\delta$, the true error E of a classifier from a family of VC dimension h, trained on N examples with training error E_{train}, satisfies

$$E \le E_{\text{train}} + \sqrt{\frac{h\big(\ln(2N/h)+1\big) + \ln(4/\delta)}{N}}.$$

The numbers it produces are loose, but the shape is the lesson: the gap between training and true error grows with capacity h and shrinks roughly as $1/\sqrt{N}$. A family that can fit anything (a 1-NN classifier, an unpruned tree) has an uninformative bound, which is why such models must be judged on held-out data.

18.2 Bias and variance of k-NN

For k-NN regression with noise variance σ^2, the variance of a prediction is about σ^2/k: averaging more neighbours reduces noise. But larger k reaches further away for neighbours, whose true values differ from the query's, so bias grows. With N points and k neighbours the model has roughly N/k *effective parameters*. On the IADS data, $k=1$ gave 78.3% validation accuracy and $k=21$ gave 90.2%: the noise-averaging gain outweighed the bias at that scale.

19 Logistic Regression as Maximum Likelihood

19.1 Deriving the loss

Model the label as a Bernoulli variable with $P(y=1 \mid x) = p(x) = \sigma(w^\top x + b)$. The likelihood of independent training labels is $\prod_i p_i^{y_i}(1-p_i)^{1-y_i}$; its negative logarithm is

$$-\ell(w,b) = -\sum_i \big[y_i \ln p_i + (1-y_i)\ln(1-p_i)\big],$$

which is exactly the cross-entropy loss. Minimising cross-entropy is therefore **maximum-likelihood estimation**. The loss is convex in (w, b), so gradient descent cannot get stuck in a poor local minimum.

19.2 Newton's method (IRLS)

Because the loss is smooth and convex, Newton's method converges in a handful of iterations: $w \leftarrow w - H^{-1}\nabla\ell$, with gradient $X^\top(p-y)$ and Hessian $H = X^\top S X$, where $S = \mathrm{diag}\big(p_i(1-p_i)\big)$. Each step is a weighted least-squares problem, hence the name *iteratively re-weighted least squares*. Statistical packages use it, and H^{-1} at the optimum gives standard errors for the weights.

19.3 Geometry and a failure mode

The decision boundary $p = 0.5$ is the hyperplane $w^\top x + b = 0$, and $|w^\top x + b|/\|w\|$ is a point's distance from it: the further away, the more confident the prediction. If the classes are **perfectly separable**, the likelihood keeps increasing as $\|w\| \to \infty$; the weights diverge and the probabilities become 0 and 1. Small defence datasets with many features are often separable by chance; L2 regularisation keeps the weights finite and the probabilities honest.

19.4 More than two classes

Multinomial (softmax) regression gives each class k a weight vector w_k and sets $P(y=k \mid x) = e^{w_k^\top x} / \sum_j e^{w_j^\top x}$. The gradient for class k is $(p_k - \mathbf{1}[y=k])\,x$ — the same simple form as the binary case.

20 From the Perceptron to the Support Vector Machine

20.1 The perceptron algorithm

With labels $y \in \{-1, +1\}$, the perceptron predicts $\mathrm{sign}(w^\top x + b)$ and, whenever an example is misclassified ($y(w^\top x + b) \le 0$), updates $w \leftarrow w + yx$, $b \leftarrow b + y$.

Worked example — perceptron by hand

Training points: $(1,1)$ and $(2,-1)$ are hostile $(+1)$; $(-1,1)$ and $(0,-2)$ are friendly (-1). Start with $w = (0,0)$, $b = 0$.
Point $(1,1)$: score 0, misclassified $\Rightarrow w = (1,1)$, $b = 1$. Point $(2,-1)$: score 2, correct. Point $(-1,1)$: score 1 but label $-1 \Rightarrow w = (2,0)$, $b = 0$. Point $(0,-2)$: score 0, misclassified $\Rightarrow w = (2,2)$, $b = -1$.
Second pass: margins $y(w^\top x + b)$ are $3, 1, 1, 5$ — all positive. Converged: $2x_1 + 2x_2 - 1 = 0$ separates the classes.

Novikoff's theorem: if the data are separable with margin γ and all points lie within radius R, the perceptron makes at most $(R/\gamma)^2$ mistakes. It finds *a* separating line, but which one depends on the order of the data. The SVM asks for the *best* one.

20.2 The SVM optimisation problem and its dual

The hard-margin SVM solves $\min \frac{1}{2}\|w\|^2$ subject to $y_i(w^\top x_i + b) \geq 1$. Introducing a Lagrange multiplier $\alpha_i \geq 0$ for each constraint gives the Lagrangian

$$L = \tfrac{1}{2}\|w\|^2 - \textstyle\sum_i \alpha_i \big[y_i(w^\top x_i + b) - 1\big].$$

Setting derivatives to zero yields $w = \sum_i \alpha_i y_i x_i$ and $\sum_i \alpha_i y_i = 0$. Substituting back gives the **dual problem**:

$$\max_{\alpha} \sum_i \alpha_i - \tfrac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \, x_i^\top x_j \quad \text{s.t. } \alpha_i \geq 0,\ \textstyle\sum_i \alpha_i y_i = 0.$$

The **Karush–Kuhn–Tucker conditions** require $\alpha_i[y_i(w^\top x_i + b) - 1] = 0$: either $\alpha_i = 0$ (the point is beyond the margin and plays no role) or the point lies exactly on the margin. Points with $\alpha_i > 0$ are the **support vectors**; the solution depends on them alone. With the soft margin, the only change is $0 \leq \alpha_i \leq C$.

Worked example — an SVM solved by hand

Hostile tracks at $(3,3)$ and $(3,4)$; a friendly track at $(1,1)$. By symmetry the boundary is perpendicular to the segment from $(1,1)$ to $(3,3)$ and passes through its midpoint $(2,2)$: $w \propto (1,1)$. The margin constraints $w^\top x + b = \pm 1$ at $(3,3)$ and $(1,1)$ give $w = (0.5, 0.5)$ and $b = -2$. Check $(3,4)$: $0.5(3) + 0.5(4) - 2 = 1.5 \geq 1$, so it is not a support vector ($\alpha = 0$). Margin width $2/\|w\| = 2/0.707 = 2.83$, exactly the distance between $(1,1)$ and $(3,3)$. From $w = \sum \alpha_i y_i x_i = \alpha_1(3,3) - \alpha_3(1,1)$ with $\alpha_1 = \alpha_3$: $\alpha(2,2) = (0.5, 0.5)$, so $\alpha_1 = \alpha_3 = 0.25$. Moving $(3,4)$ anywhere outside the margin would not change the classifier at all.

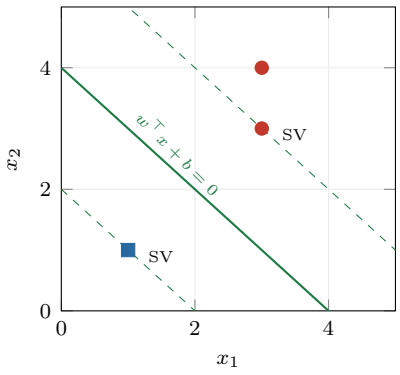


Figure C1: the hand-solved SVM. Dashed lines are the margins $w^\top x + b = \pm 1$; SV marks the support vectors.

20.3 Kernels

The dual uses the data only through inner products $x_i^\top x_j$. Replacing them by a **kernel** $K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$ trains a linear SVM in the feature space ϕ without ever computing ϕ.

Worked example — a kernel is a hidden inner product

For the polynomial kernel $K(x,z) = (x^\top z + 1)^2$ with $x = (1,2)$, $z = (3,1)$: $x^\top z = 5$, so $K = 36$. The explicit six-dimensional map $\phi(v) = (1, \sqrt{2}v_1, \sqrt{2}v_2, v_1^2, v_2^2, \sqrt{2}v_1 v_2)$ gives $\phi(x) = (1, 1.414, 2.828, 1, 4, 2.828)$ and $\phi(z) = (1, 4.243, 1.414, 9, 1, 4.243)$, whose inner product is also 36. The kernel did the work of the six-dimensional map with one multiplication and a square. The RBF kernel $K = \exp(-\gamma\|x - z\|^2)$ corresponds to an infinite-dimensional map; with

$\gamma = 0.5$ these two points give $e^{-0.5\times 5} = 0.082$.

Mercer's condition: a function is a valid kernel if every kernel matrix it produces is symmetric and positive semi-definite. Sums and products of valid kernels are valid, so domain-specific kernels (for strings, graphs, time series) can be built from simple ones.

20.4 The SVM as a regularised loss

The soft-margin SVM is equivalent to minimising $\frac{1}{N}\sum_i \max\big(0, 1 - y_i f(x_i)\big) + \lambda\|w\|^2$: hinge loss plus L2 regularisation. Logistic regression replaces the hinge by the logistic loss $\ln(1 + e^{-yf})$. The two give similar boundaries; the hinge ignores points that are confidently correct, which is why only support vectors matter, while logistic regression gives calibrated probabilities. SVM scores are not probabilities; Platt scaling fits a sigmoid to them on validation data when probabilities are needed.

21 Trees and Ensembles in Depth

21.1 The CART algorithm

Algorithm — growing a classification tree (CART)

1. At a node, for every feature sort the values and consider thresholds midway between consecutive distinct values.
2. For each candidate split compute the weighted impurity of the two children; choose the split with the largest impurity decrease.
3. Recurse on each child until a stopping rule holds (maximum depth, minimum samples per leaf, or no impurity decrease).
4. Label each leaf with its majority class (or class proportions as probabilities).

The search is greedy: the best split now may not lead to the best tree. Trees handle mixed feature types and need no scaling, but small changes in data can change the top split and therefore the whole tree — the instability that ensembles exploit.

21.2 Cost-complexity pruning

Grow a large tree, then prune it back by minimising $R_\alpha(T) = R(T) + \alpha|T|$, where $R(T)$ is the training error and $|T|$ the number of leaves. For each internal node, compare the subtree below it with collapsing it to a leaf.

Worked example — when to prune

A subtree with 5 leaves has error 0.10; collapsing it to one leaf gives error 0.16. The two are equally good when $0.10 + 5\alpha = 0.16 + \alpha$, i.e. $\alpha = 0.015$. For any complexity penalty above 0.015 per leaf the subtree is pruned. Increasing α prunes progressively, producing a sequence of trees from which cross-validation picks one.

Regression trees use the same procedure with variance (squared error) as the impurity and the leaf mean as the prediction.

21.3 How much does bagging help?

If each of B trees has prediction variance σ^2 and any two trees have correlation ρ, the variance of their average is

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

With $\sigma^2 = 1$ and $\rho = 0.3$: one tree 1.0; ten trees 0.37; a hundred trees 0.307 — and no number of trees gets below $\rho\sigma^2 = 0.3$. Reducing the correlation to $\rho = 0.05$ (by sampling a random subset of features at each split) brings 100 trees down to 0.06. This formula is the whole design rationale of random forests: decorrelate the trees, then average many of them.

21.4 Gradient boosting and the XGBoost split gain

Boosting builds an additive model $F_M(x) = \sum_m \eta\, T_m(x)$, each tree fitted to the negative gradient of the loss at the current predictions (for squared error, the residuals). XGBoost uses first and second derivatives of the loss — g_i and h_i for each example — and regularises each tree. For a candidate split of a node into left and right sets, the gain is

$$\text{Gain} = \tfrac{1}{2}\Big[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\Big] - \gamma_{\text{split}},$$

with G and H the sums of g_i and h_i in each set, and optimal leaf values $w = -G/(H+\lambda)$.

Worked example — one XGBoost split

Four examples with squared-error gradients $g = (-2, -1, 1, 3)$ and $h = 1$ each; $\lambda = 1$, $\gamma_{\text{split}} = 0$. Splitting into $\{-2, -1\}$ and $\{1, 3\}$: $G_L = -3$, $G_R = 4$, $H_L = H_R = 2$. Gain $= \tfrac{1}{2}[9/3 + 16/3 - 1/5] = 4.07$. Leaf values: left $+1.0$, right -1.33 — each leaf moves its predictions against its gradient, i.e. toward the targets, shrunk by λ.

21.5 Stacking and voting

Hard voting takes the majority class of several models; **soft voting** averages their probabilities. **Stacking** trains a second-level model on the out-of-fold predictions of the first-level models, learning when to trust which. Ensembles of diverse models (a forest, an SVM and a neural network) often beat each member, but the gain shrinks when members make the same errors.

22 Probabilistic Classifiers in Depth

22.1 Gaussian Naive Bayes worked fully

Worked example — bomber or fighter from speed and radar cross-section

Training data give: bombers (prior 0.3) speed $\sim \mathcal{N}(850, 100^2)$ km/h, RCS $\sim \mathcal{N}(20, 8^2)$ m^2; fighters (prior 0.7) speed $\sim \mathcal{N}(1100, 250^2)$, RCS $\sim \mathcal{N}(5, 3^2)$. A track flies at 950 km/h with RCS 12 m^2.
Likelihoods: bomber speed density 2.42×10^{-3}, RCS 3.03×10^{-2}; fighter speed 1.33×10^{-3}, RCS 8.74×10^{-3}.
Unnormalised posteriors: bomber $0.3 \times 2.42 \times 10^{-3} \times 3.03 \times 10^{-2} = 2.20 \times 10^{-5}$; fighter $0.7 \times 1.33 \times 10^{-3} \times 8.74 \times 10^{-3} = 8.16 \times 10^{-6}$.
Normalising: $P(\text{bomber} \mid x) = 0.73$. Using speed alone the answer would have been 0.44 (fighter more likely, because of the prior): the RCS evidence reversed the decision. If speed

and RCS were strongly correlated, multiplying their likelihoods would double-count evidence and push the probability too far — the over-confidence that made Naive Bayes the weakest IADS model.

22.2 Text classification and Laplace smoothing

For intelligence reports represented as word counts, **multinomial Naive Bayes** estimates $P(\text{word} \mid \text{class})$ from word frequencies. A word never seen in a class would get probability zero and veto that class entirely, so counts are smoothed: $P(w \mid c) = (n_{w,c} + 1)/(n_c + V)$, where V is the vocabulary size (**Laplace smoothing**). Naive Bayes remains a strong, fast baseline for triaging large volumes of text.

22.3 Discriminant analysis

Linear discriminant analysis assumes Gaussian classes with a shared covariance matrix, giving linear boundaries; **quadratic discriminant analysis** allows each class its own covariance, giving curved boundaries. Unlike Naive Bayes they model correlations between features, at the cost of estimating covariance matrices.

23 Multi-class Evaluation in Depth

Worked example — a three-class confusion matrix

Rows are true classes, columns predictions, for fighters (F), bombers (B) and UAVs (U):

true \ pred	F	B	U	total
F	**50**	5	5	60
B	10	**30**	0	40
U	2	3	**15**	20
total	62	38	20	120

Accuracy $= 95/120 = 0.792$. Precision: F $50/62 = 0.806$, B $30/38 = 0.789$, U $15/20 = 0.750$. Recall: F $50/60 = 0.833$, B $30/40 = 0.750$, U $15/20 = 0.750$. F_1: 0.820, 0.769, 0.750; **macro-F_1** (unweighted mean) $= 0.780$. **Micro-averaging** pools all decisions and, for single-label problems, equals accuracy (0.792). Macro-averaging gives the rare UAV class equal weight, which is usually what a defence evaluation should do.

Cohen's kappa corrects accuracy for agreement expected by chance: $p_e = \sum_k (\text{row}_k \times \text{col}_k)/N^2 = (60 \cdot 62 + 40 \cdot 38 + 20 \cdot 20)/120^2 = 0.392$, so $\kappa = (0.792 - 0.392)/(1 - 0.392) = 0.658$ — "substantial" agreement, more modest than the raw accuracy suggests.

24 Satellite and SAR Image Interpretation

Image interpretation is the most mature supervised-learning application in defence. Satellites and aircraft produce more imagery than analysts can inspect; classifiers and detectors triage it so that analysts look first where it matters.

24.1 The four tasks

Task	Output	Example
Classification	one label per image chip	"this chip contains a T-72-class tank"
Detection	a box and a label for every object	locate all vehicles in a 10 km^2 scene
Segmentation	a label for every pixel	map roads, buildings, water, vegetation
Change detection	what differs between two dates	new revetments at an airfield since last week

24.2 How detectors work

A CNN backbone (see the deep-learning section of the Lecture 1 text) turns the image into feature maps. **Two-stage detectors** (the R-CNN family) first propose candidate regions, then classify and refine each; they are accurate but slower. **One-stage detectors** (YOLO, SSD, RetinaNet) predict boxes and classes directly from a grid of anchor positions in one pass; they are fast enough for real-time video. Overlapping duplicate boxes are removed by **non-maximum suppression**: keep the highest-scoring box and discard others that overlap it by more than a threshold.

24.3 Measuring detection quality: IoU and average precision

A predicted box counts as correct if its **intersection over union** with a true box exceeds a threshold (commonly 0.5):

$$\mathrm{IoU} = \frac{\mathrm{area}(B_{\mathrm{pred}} \cap B_{\mathrm{true}})}{\mathrm{area}(B_{\mathrm{pred}} \cup B_{\mathrm{true}})}.$$

Worked example — IoU of two boxes

True box with corners $(10, 10)$–$(50, 50)$ (area 1,600 pixels). A prediction at $(15, 15)$–$(55, 55)$ overlaps in a 35×35 square: intersection 1,225, union $1{,}600 + 1{,}600 - 1{,}225 = 1{,}975$, IoU $= 0.62$ — a hit at the 0.5 threshold. A prediction at $(30, 30)$–$(70, 70)$ overlaps only $20 \times 20 = 400$: IoU $= 400/2{,}800 = 0.14$ — a miss, even though the boxes visibly touch.

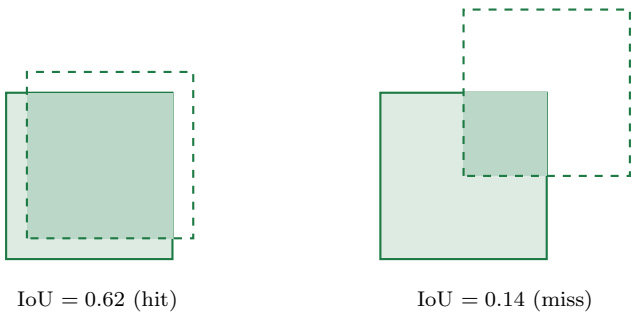


Figure C2: true box (solid) and predicted box (dashed); the shaded overlap is the intersection.

Worked example — average precision for one class

A scene contains 5 vehicles. The detector's 8 detections, sorted by confidence, are: hit, hit, false, hit, false, hit, false, false. Running precision and recall: $(1.00, 0.2)$, $(1.00, 0.4)$, $(0.67, 0.4)$, $(0.75, 0.6)$, $(0.60, 0.6)$, $(0.67, 0.8)$, $(0.57, 0.8)$, $(0.50, 0.8)$. Using the interpolated precision (the best precision at that recall or higher) at each recall step: AP $= 0.2(1.00) + 0.2(1.00) + 0.2(0.75) + 0.2(0.67) = 0.68$. One vehicle was never found, so recall never reaches 1 and

the last 0.2 of the curve contributes nothing. **mAP** averages AP over classes (and, in some benchmarks, over IoU thresholds from 0.5 to 0.95).

24.4 Practical issues specific to overhead imagery

- **Small objects and huge scenes.** A vehicle may be 10×20 pixels in a 30,000×30,000 image. Scenes are tiled into overlapping chips and detections are merged across tile boundaries.
- **Arbitrary orientation.** Seen from above, objects appear at every rotation; rotated boxes and rotation augmentation are used.
- **SAR is not a photograph.** Synthetic-aperture radar images show speckle noise, layover and shadow, and a vehicle's signature changes strongly with the look (aspect and depression) angle. Public benchmarks such as MSTAR (ten vehicle classes) show near-perfect accuracy under matched conditions and sharp drops when depression angle or configuration changes — distribution shift again.
- **Transfer learning.** Backbones pre-trained on large image collections are fine-tuned on a few thousand labelled chips; self-supervised pre-training on unlabelled imagery from the same sensor works better still.
- **Change detection.** Simple image differencing after co-registration flags change but also flags shadows, seasons and sensor differences; learned Siamese networks compare deep features of the two dates and are more robust.
- **Deception.** Decoys, camouflage nets and adversarial patterns target exactly these detectors. Multi-sensor confirmation (EO, SAR, SIGINT) is the defence.

Defence & wargaming application — imagery triage

A useful operating concept: the detector runs on every scene and ranks chips by threat-class score; analysts review the top-ranked chips first and confirm or reject; their decisions become new labels; the model is retrained regularly. Success is measured by analyst hours saved and threats found earlier, at a stated recall, not by benchmark mAP.

25 Knowing When Not to Answer

25.1 Open-set recognition

A closed-set classifier must assign every input to one of its training classes. A new radar type will therefore be confidently mislabelled as the nearest known type. **Open-set recognition** adds the option "none of the above": reject inputs whose maximum class probability is low, whose feature vector is far from every class centre, or whose anomaly score (Lecture 5) is high. The rejection threshold is set on validation data that include examples held out from the known classes.

25.2 The reject option

Even among known classes, some inputs are ambiguous. Chow's rule rejects when the top posterior is below $1 - c_r$, where c_r is the cost of referring the case to a human relative to the cost of an error. Rejection lowers coverage but raises accuracy on the cases the machine does decide; reporting accuracy together with coverage gives a fair picture.

26 Explaining a Classifier's Decisions

26.1 Permutation importance

Shuffle one feature's values in the validation set and measure how much performance drops. A large drop means the model relies on the feature. Correlated features share importance, so a feature can appear unimportant because a correlated partner carries the same information.

26.2 Shapley values

Shapley values, from cooperative game theory, divide a prediction among the features fairly: each feature receives its average marginal contribution over all orders in which features could be added.

Worked example — Shapley values with two features

Baseline threat probability (no features known) 0.30; knowing only EW activity gives 0.70; knowing only speed gives 0.40; knowing both gives 0.90. EW's contribution is $0.70-0.30 = 0.40$ if added first and $0.90 - 0.40 = 0.50$ if added second; its Shapley value is the average, 0.45. Speed's is $\frac{1}{2}[(0.40 - 0.30) + (0.90 - 0.70)] = 0.15$. The two sum to $0.60 = 0.90 - 0.30$: the explanation accounts exactly for the difference between this prediction and the baseline. SHAP software approximates this computation efficiently for many features.

Pitfall — explanations are about the model

A Shapley value says how the *model* used a feature, not whether the feature causes the outcome. If the model has learned a shortcut, the explanation will faithfully report the shortcut — which is exactly why explanations are useful for auditing.

27 Further Self-Test Questions

Check your understanding — Part C

(1) What is the VC dimension of linear classifiers in three dimensions? (2) Why does moving a non-support vector not change an SVM? (3) With $\rho = 0.3$, what is the smallest variance any number of averaged trees can reach (with $\sigma^2 = 1$)? (4) Compute IoU for boxes $(0, 0)$–$(10, 10)$ and $(5, 0)$–$(15, 10)$. (5) A classifier has 95% accuracy at 80% coverage with a reject option. What does that mean operationally?

Answers. (1) 4. (2) Its multiplier α_i is zero, and $w = \sum \alpha_i y_i x_i$ does not involve it. (3) 0.3. (4) Intersection $5 \times 10 = 50$, union $100 + 100 - 50 = 150$, IoU $= 0.33$. (5) The machine decides 80% of cases automatically with 95% accuracy; the remaining 20% go to analysts, whose workload must be planned for.

28 Implementation Notes

```
from sklearn.model_selection import StratifiedKFold, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
pipe = Pipeline([("scale", StandardScaler()),
    ("rf", RandomForestClassifier(class_weight="balanced", random_state=0))])
grid = {"rf__n_estimators":  [200, 500], "rf__max_features":  ["sqrt", 0.3]}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
search = GridSearchCV(pipe, grid, cv=cv, scoring="f1_macro").fit(X_train, y_train)
```

Putting scaling inside the pipeline ensures it is re-fitted on each training fold, preventing leakage; scoring by macro-F1 rather than accuracy protects the rare classes. For SMOTE use the `imblearn` pipeline so that oversampling happens inside each fold.

29 Self-Test Questions

Check your understanding — five questions

1. Why is one-hot encoding preferred to integer codes for polarisation? **2.** A tree has training accuracy 99.8% and test accuracy 88.7%. What is happening and what would you change? **3.** Compute α_m for an AdaBoost learner with weighted error 0.3. **4.** A miss costs 50 times a false alarm. What probability threshold should trigger an alert? **5.** Two classifiers score 96.4% and 96.7% on 1,275 tracks. Can you claim the second is better?

Answers. 1: integers impose a false order and spacing. 2: overfitting (high variance); limit depth, prune (α), or use a forest. 3: $\frac{1}{2}\ln(0.7/0.3) = 0.42$. 4: $1/51 \approx 0.02$, provided probabilities are calibrated. 5: no — the difference is within sampling error; use McNemar's test on the paired predictions.

30 Classification Across the Disciplines

- **Radar and psychology.** The ROC curve originated with WWII radar operators deciding "target or noise"; signal-detection theory (Peterson, Birdsall & Fox, 1954) then entered psychophysics (Tanner & Swets, 1954). Every IADS alert threshold is a point on an ROC curve.
- **Medicine.** Cancer screening trades sensitivity against false recalls, choosing the operating point by the cost of a missed case.
- **Engineering.** Vibration signatures classify bearing and gearbox faults before failure — the basis of helicopter health and usage monitoring.
- **Psychology.** Meehl's *Clinical versus Statistical Prediction* (1954) found that simple actuarial rules matched or beat expert judgement in most comparisons.
- **Statistics.** Fisher's iris data (1936) introduced linear discriminant analysis, the ancestor of linear classifiers.

31 Summary and Glossary

Logistic regression is the interpretable linear baseline; kNN is flexible but slow and scale-sensitive; trees are fully auditable; random forests give the best accuracy on tabular data; boosting gives speed and strong minority recall; SVMs give principled margins and kernels; Naive Bayes is a fast baseline that fails with correlated features. Always evaluate per class, on the operational base rate, with the cost of each error in mind.

Bagging	Averaging models trained on bootstrap samples to reduce variance.
Boosting	Sequentially adding models that fit the previous ensemble's residuals.
Calibration	Agreement between predicted probabilities and observed frequencies.
Cross-entropy	Negative log-likelihood loss for probabilistic classifiers.
Gini impurity	$1 - \sum p_c^2$: probability of mislabelling a random draw from a node.
Kernel	Function computing an inner product in an implicit feature space.
Margin	Distance from the decision boundary to the nearest training points.
Odds ratio	e^{w_j}: multiplicative change in odds per unit of feature j.
Out-of-bag error	Error on examples not in a tree's bootstrap sample.
SMOTE	Synthetic minority oversampling by interpolation.
Support vector	Training point lying on or inside the SVM margin.